

Block Allocation Engine — Algorithm Architecture

This document covers the full algorithmic pipeline: template reconstruction, demand forecasting, stochastic optimization, and explainability. It is the technical reference for the algo layer. For API contracts and input/output schemas see the companion API spec. For the app-side UX see the companion PRD.

Table of Contents

1. Pre-Layer: Template Reconstruction
2. Layer 1: Stochastic Demand Forecasting
3. Layer 2: Stochastic MIP + Pareto Frontier
4. Layer 3: Explainability

1. Pre-Layer: Template Reconstruction

Status: Implemented. The upgraded pipeline adds three new outputs on top of the existing implementation: `exception_rate_per_series`, `phase_labels`, and `phase_offset`. These feed the demand forecasting layer.

1.1 What It Does

The template reconstruction step is the entry point of the entire pipeline. Before any optimization can happen, the engine needs to understand the **structural schedule pattern** — does the OR schedule repeat every week, every 2 weeks, every 4 weeks?

This step answers that question from historical block data alone, with no prior assumptions. It then reconstructs the repeating template by assigning the dominant provider to each recurring slot, and records all weeks where the actual assignment deviated from that pattern.

The reconstructed template serves two purposes:

- **Warm-start for the optimizer** — the MIP starts from the current dominant assignment rather than a blank slate.
- **Phase-alignment reference for demand forecasting** — Layer 1 groups historical demand data by rotation phase before building features, so trailing windows respect the true structural cycle rather than arbitrary calendar weeks.

1.2 Inputs

From historical database query:

Field	Type	Description
Blocks	List<Block>	Every block occurrence in the historical range: site, room, day, time slot, week, provider ID or Open/Closed, manual early release
Cases	List<Case>	Case ID, provider ID, block case minutes, turnover time, open block boolean
Providers	List<Provider>	Provider ID, name, service line, exclusive sites
Locations	List<Location>	ID, name, parent location ID, type

Parameters:

Parameter	Type	Default	Description
historical_range	Tuple(start, end)	Last 3 months	Date range to reconstruct template from
current_date	date	—	Anchor for relative date calculations
template_weeks	int	0	0 means run BIC detection automatically. Greater than 0 means force that period length

1.3 Algorithm: Period Detection via BIC

The algorithm tests every candidate rotation period.

$$T \in 1, 2, \dots, \min(13, H)$$

Here, H is the number of historical weeks.

For each candidate period T , the algorithm asks:

If the schedule truly repeats every T weeks, how well does that explain the historical data?

Step 1 — Phase Grouping

For each candidate period T , group all historical weeks by their rotation phase.

$$W_{T,p} = \{w \mid 0 \leq w \leq H - 1, w \bmod T = p\}$$

Each week belongs to exactly one phase.

$$p \in 0, 1, \dots, T - 1$$

For example, under a 2-week cycle:

- Phase 0 contains weeks with $w \bmod 2 = 0$.
- Phase 1 contains weeks with $w \bmod 2 = 1$.

Step 2 — Modal Template Value

A **cell** is one unique OR slot — one combination of room, day, and time.

For each cell c , phase p , assignment value v , and candidate period T , define the count of how many times value v appears.

$$N_{c,p,T} = |\{w \in W_{T,p} \mid x_c(w) = v\}|$$

The modal template value is the most frequently observed assignment.

$$m_{c,p,T} = \arg \max_v N_{c,p,T}$$

This is the **dominant provider** for that cell in that phase.

All other providers who held the cell in that phase are exceptions.

Step 3 — Deviation Count

A deviation is any week where the actual provider is different from the modal provider for that cell and phase.

For each cell and phase, the deviation count is:

$$d_{c,p,T} = |W_{T,p}| - \max_v N_{c,p,T}$$

The total number of deviations across all cells and phases is:

$$D(T) = \sum_c \sum_{p=0}^{T-1} d_{c,p,T}$$

Lower $D(T)$ means the period T explains the historical data better — fewer weeks deviated from what the template would predict.

Step 4 — BIC Scoring

The BIC objective balances goodness-of-fit against model complexity.

A longer rotation period can fit the historical data better, but it costs more to describe.

$$BIC(T) = T|\mathcal{C}| \ln(V) + D(T) \ln(H|\mathcal{C}|V)$$

Where:

- $|\mathcal{C}|$ = number of cells, meaning unique room × day × time combinations.
- V = alphabet size, meaning the number of possible assignees per cell: providers + Open + Closed.
- H = total number of historical weeks.

The first term is the **template cost**.

$$T|\mathcal{C}| \ln(V)$$

This grows linearly with the candidate period T , because longer periods require more template slots to specify.

The second term is the **deviation cost**.

$$D(T) \ln(H|\mathcal{C}|V)$$

This penalizes misfit. Each week where the actual assignment differs from the modal assignment increases the deviation cost.

Step 5 — Select Optimal Period

The selected period is the candidate period with the smallest BIC score.

$$T^* = \arg \min_T BIC(T)$$

If `template_weeks > 0`, the algorithm skips BIC and forces the value of `T*` to equal `template_weeks`.

The phase offset sweep still runs even when the period is manually forced.

Step 6 — Phase Offset Detection

After `T*` is determined, all candidate offsets are tested.

$$k \in 0, 1, \dots, T^* - 1$$

Offset `k` means historical week `w` maps to template phase:

$$p_k(w) = (w + k) \bmod T^*$$

The winning offset is the offset that minimizes the total number of deviations.

$$k = \arg \min_k \sum_c \sum_{p=0}^{T^*-1} d_{c,p,T^*,k}$$

This ensures the template phases align with the actual cycle structure in the data, not with an arbitrary starting point.

1.4 Block Clustering: 2-Hour Margin of Error

Raw historical data contains slight time variations week to week.

For example, a block nominally at 07:00 might appear as 07:05 in some weeks because of EHR recording differences.

Canonical-time clustering runs per site, day, and room group after collecting per-week slots.

Any start-end pair within a 2-hour margin of another is snapped to the most frequently observed pair in that cluster.

For start times:

$$|start_i - start_j| \leq 2 \text{ hours}$$

For end times:

$$|end_i - end_j| \leq 2 \text{ hours}$$

This merges fragmented cells back into one dense cell.

Why 2 hours: Empirically validated against ground truth templates — a 2-hour window correctly groups blocks to match the true template without over-merging distinct time slots.

1.5 Dominant Provider Assignment

For each template block, meaning each cell × phase pair, the dominant provider is the modal value.

$$m_{c,p,T^*}$$

Any week where a different provider held that block is recorded as an **exception**.

Let `ab,w` be the actual provider assigned to block series `b` in week `w`.

The exceptions list is:

$$E = (b, w, a_{b,w}) \mid x_b(w) \neq m_{b,p(w),T^*}$$

The exception rate per block series is:

$$r_b = \frac{|w \mid x_b(w) \neq m_{b,p(w),T^*}|}{H}$$

In the API, this value is exposed as:

```
exception_rate_per_series[b] = r_b
```

A high exception rate signals a contested slot — the block does not reliably belong to one provider.

This is a meaningful signal of demand uncertainty and is passed to the demand forecasting layer as a new feature.

1.6 Outputs

Output	Type	Description
<code>T*</code>	<code>int</code>	Optimal rotation period in weeks
<code>phase_offset</code>	<code>int</code>	Winning phase offset
<code>block_template</code>	<code>List<Block></code>	Dominant provider per cycling slot across all <code>T*</code> rotation weeks

Output	Type	Description
<code>exceptions_list</code>	<code>List<(block_id, date, provider_id)></code>	All historical weeks where actual provider differs from dominant provider
<code>exception_rate_per_series</code>	<code>Dict<block_id, float></code>	Fraction of weeks with non-modal assignment per series
<code>phase_labels</code>	<code>Dict<week, int></code>	Maps each historical week to its rotation phase
<code>utilization_per_provider</code>	<code>Dict<provider_id, float></code>	Historical utilization per provider for display
<code>warnings</code>	<code>List<string></code>	Data validation warnings

The phase labels map each historical week to one of the following phases:

$$p \in 0, \dots, T^* - 1$$

1.7 Data Validation

Check	Type	Message
No same room has 2 blocks overlapping in time	Error	—
All providers assigned to blocks exist in providers list	Error	—
Cases has no matching Block ID	Error	—
Block provider ID is empty, treated as open block	Warning	—
Block location ID type is empty	Warning	—
Block service line is empty	Warning	—
Case provider ID is empty	Warning	—
Cases' Block ID is empty, treated as open block	Warning	—
Site for location ID is empty	Warning	—

1.8 What the Upgraded Pipeline Adds

The existing implementation produces `T*`, `block_template`, and `utilization_per_provider`.

The upgraded pipeline adds three new outputs that feed the downstream layers.

`exception_rate_per_series` → feeds demand forecasting as a new feature

High exception rate means the block is contested historically.

The demand forecasting model uses this as a signal that predictions for this slot carry more uncertainty.

The XGBoost model and Bayesian variance model both receive this as a feature.

The Bayesian model will produce a wider posterior standard deviation for high-exception series.

$$\sigma_{p,d}$$

phase_labels → feeds demand forecasting for phase-aligned history

Before building trailing-window features, the demand forecasting layer groups historical observations by rotation phase rather than by raw calendar week.

A provider on a 2-week rotation who behaves differently in odd weeks and even weeks would have their history blurred by naive 4-week averages.

Phase-aligned windows respect the true structural cycle.

exceptions_list → feeds optimizer for series consistency scores

The optimizer uses the full exceptions list to compute a consistency score.

$$s_{p,r}$$

In the API and implementation, this corresponds to:

`consistency_score[p][series]`

This score is the fraction of series instances where provider `p` is available.

It becomes one of the seven objective components in the Pareto optimization.

1.9 Utilization Calculation

Block utilization per provider

Block utilization per provider measures how efficiently each provider uses their allocated block time.

Let:

- `CaseTime_p,b` = case minutes for provider `p` in block `b`
- `Turnover_p,b` = turnover minutes for provider `p` in block `b`

- $AllocatedBlockTime_{p,b}$ = allocated block minutes
- $EarlyRelease_{p,b}$ = early manual release duration

Then provider utilization is:

$$U_p = \frac{\sum_{b \in B_p} (CaseTime_{p,b} + Turnover_{p,b})}{\sum_{b \in A_p} (AllocatedBlockTime_{p,b} - EarlyRelease_{p,b})}$$

Here:

- B_p = blocks with historical case activity for provider p
- A_p = assigned blocks for provider p

Projected demand

Each quantity is projected from the historical window to the optimization window by scaling with alpha.

Equation block:

$$\alpha = \frac{T_{opt}}{T_{hist}}$$

Projected case time for provider p :

$$CaseTimeHat_p = \alpha \sum_{c \in C_p} CaseTime_c$$

Projected turnover time for provider p :

$$TurnoverHat_p = \alpha \sum_{c \in C_p} Turnover_c$$

Projected early release duration for provider p :

$$EarlyReleaseHat_p = \alpha \sum_{b \in B_p} EarlyRelease_b$$

Assumption: Turnover and early release can be extrapolated from the historical range to the optimization range using the same alpha scaling.

Block time required

The block minutes to allocate for provider `p` in order to hit their target utilization, adjusted for expected early release, is:

$$BlockTimeRequired_p = \frac{CaseTimeHat_p + TurnoverHat_p}{TargetUtilization_p} + EarlyReleaseHat_p$$

1.10 API Endpoint

Endpoint: `/block-allocation-baseline-fetch`

Transport: SQS Lambda, async. May take 10–30 seconds for large historical ranges.

Request payload

```
{
  "historical_range": {"start": "2025-10-01", "end": "2026-01-01"},
  "current_date": "2026-01-15",
  "template_weeks": 0,
  "blocks": [...],
  "cases": [...],
  "providers": [...],
  "locations": [...]
}
```

Response payload

```
{
  "block_template": [...],
  "providers": [...],
  "T_star": 2,
  "phase_offset": 0,
  "phase_labels": {"0": 0, "1": 1, "2": 0},
  "exception_rate_per_series": {"BLK-001": 0.08, "BLK-002": 0.23},
  "exceptions_list": [...],
  "warnings": [...]
}
```